Topic:	***Interprocess communication: Send-Receive-Reply***
Tools:	***Momentics - QNX Development IDE,*** ***QNX Neutrino Target system***
Required skills:	***Basic knowledge of issues related to programming in C language and real-time systems***

Introduction:

Communication and synchronization of processes

The core of the QNX system implements four interprocess communication mechanisms: synchronous message transfer over established communication channel, synchronization using semaphores, asynchronous transfer of deposits (*fixed messages*) through deposit processes (*deposit*, *proxy*) and event signaling (*signal*). All mechanisms - except semaphores - can be used for communication of processes running on different network nodes. Additionally, appropriate system processes enable buffered transfer of data between processes through pipes and FIFOs and message queues.

Messages

The basic mechanism of process communication is the exchange of a pair of messages (message-reply), carried out according to the meeting schedule. The basic steps for exchanging messages are performed by three system functions: MsgSend(), MsgReceive(), and MsgReply(). The sending process must always know the identity of the recipient whose identifier is one of the parameters of the MsgSend() function. The process receiving the message does not need to know the identity of the sender - depending on the method of calling the MsgReceive() function, messages can be received from the indicated sender or from any process. If many messages from different senders to the same recipient are sent at the same time, all sending processes are suspended. By implication, the queue of waiting processes (and their messages) is handled according to the FIFO algorithm. However, this algorithm can be changed to priority by setting the appropriate system tags. All messages are routed directly between processes, i.e. the body of the message is copied from the sender's address space to the receiver's address space. During the exchange of messages, however, there is a certain buffer between the processes which allows the process receiving the message and sending the reply to receive or transmit the message in fragments, without the need to accumulate the entire content in one table.

Messages exchanging

Transferring a message between processes is the transfer of a certain number of bytes between these processes according to a fixed protocol. The transmission of the message is an atomic operation.

The ability to pass messages between processes is a fundamental property of the QNX system. In the QNX system, we distinguish the following actions related to sending messages between P1 and P2 processes:

1. Sending a message via P1 to P2
2. Receipt of the message by P2
3. Sending a reply from P2 to P1.

Message sending

The message is sent to process P by executing the MsgSend function. The prototype of the MsgSend function is given below.

```
int MsgSend( int coid,
             const void* smsg,
             int sbytes,
             void* rmsg,
             int rbytes );
```

The meaning of the parameters of the above function is as follows:

- coid - connection id
- smsg - sending data buffer address
- rmsg - received data buffer address
- sbytes - number of bytes to send
- rbytes - max. number of bytes to be received

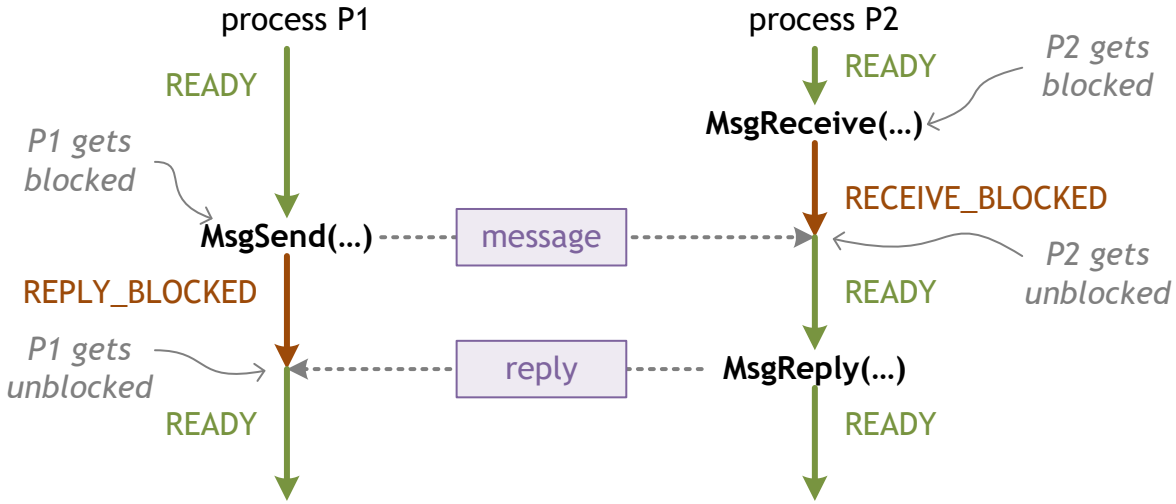
The function returns:

- value of the Status field in MsgReply() - success,
- -1 - error

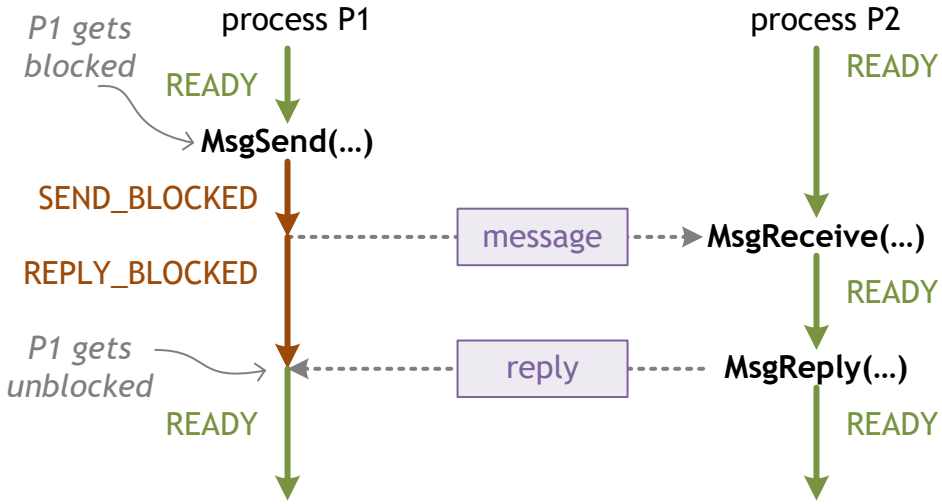
The operation of the MsgSend function is as follows:

1. The smsg message is sent to the target process, after which the current process is blocked.
2. The sending process is blocked until the target process receives the sent message and sends a reply.
3. When a response is received, it is placed in the rmsg buffer and the current process is resumed.

Exchange of messages between P1 and P2 processes.
The MsgReceive(...) function precedes the MsgSend(...)



Exchange of messages between P1 and P2 processes.
The MsgSend(...) function precedes the MsgReceive(...)



Message receiving

The MsgReceive() function is used to receive messages by the P process.

```
int MsgReceive( int chid,
                void * msg,
                int bytes,
                struct _msg_info * info );
```

The meaning of the parameters of the above MsgReceive function is as follows:

- chid - channel ID
- msg - received data buffer address
- bytes - max. number of bytes to be received
- info - additional information about the message

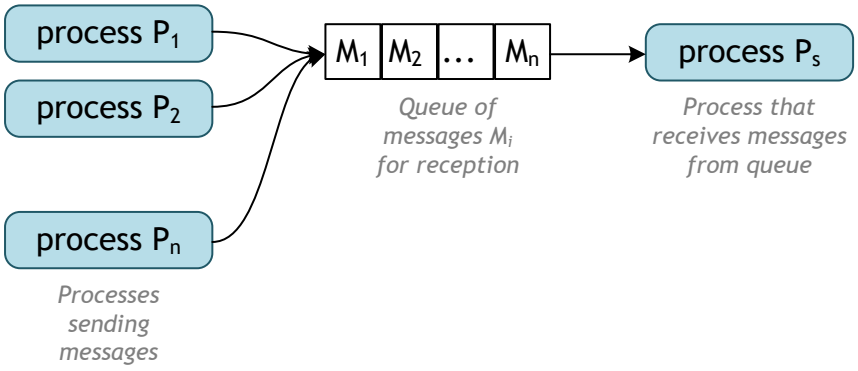
The function returns:

- >0 – ID of the process that sent the message
- 0 – in case of receiving pulse
- -1 – error

When there is a need to receive a message, the MsgReceive function should be performed. The operation of the Receive function depends on the existence of messages that have previously been sent to process P by other processes. If so, they must wait in queue Q for missed messages.

The operation of the MsgReceive function is as follows:

1. When there are messages waiting in the Q queue, the first of them is removed from the Q queue and placed in the msg buffer. The function returns the ID of the process that sent this message. The system unblocks the process sending the message. The current process is not blocked.
2. When queue Q is empty, process P is blocked until such a message arrives. A blocked process is in the RECEIVE_BLOCKED state. You can reorder this queue to order according to the priority of the processes sending messages, but the priority organization of the queue entails the possibility of starving the processes.



Replying the message

```
int MsgReply( int rcvid,
              int status,
              const void* msg,
              int size );
```

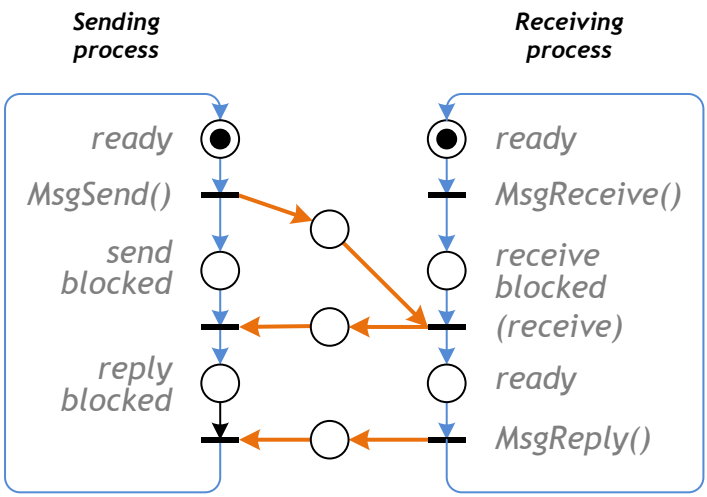
The meaning of the parameters of the above MsgReply function is as follows:

- rcvid - identifier returned by the MsgReceive() function
- status - the value that will be returned by the MsgSend() function
- msg - sending data buffer address
- size - number of bytes to send

The function returns:

- 0 – successful reply delivery
- -1 - error (error code is stored in errno)

The figure below shows the message exchange between P1 and P1 processes depending on the time.



Names

The P1 process can send a message to P2 as long as it knows its PID and the identifier of the channel created by P2. Without additional mechanisms, only processes that are in a parent / child relationship can communicate in this way. A naming mechanism has been introduced in QNX to allow unbound processes to communicate. Processes can **register** their names in the system. The names are plain strings. Other processes may ask the system for the process ID with the name of the process. This mechanism also works over the network and is called localization.

Name **registration**

The process registers the name and creates a communication channel by executing the function:

```
name_attach_t * name_attach( dispatch_t * dpp,
                             const char * path,
                             unsigned flags );
```

The meaning of the parameters of the above **name_attach** function is as follows:

- dpp – data structure for the name being created (if NULL, it will be created)
- path – the name path to be registered
- flags – additional flags

The function returns:

- pointer to the name_attach_t structure,
- -1 - error

Finding a process with a given name

The identifier of the connection created to the process with the given name is obtained by calling a function:

```
int name_open( const char * name,
               int flags );
```

The meaning of the parameters of the above **name_open** function is as follows:

- name - the name of the process being located
- flags – optional flags

The function returns:

- 0 – ID of the created connection
- -1 - error

Name deregistration

The name is deleted by calling the function:

```
int name_detach( name_attach_t * attach,
                 unsigned flags );
```

The meaning of the parameters of the above **name_detach** function is as follows:

- attach – structure returned by name_attach

The function removes the name from the namespace.

The function returns:

- 0 - when deregistration was successful
- -1 - error

Purpose of the exercise

The aim of the exercise is to get acquainted with synchronous interprocess communication (IPC) in the QNX system and the implementation of programs communicating on one network node and through the network.

Exercise program

1. Implementation of the message receiving program (server using the MsgReceive() and MsgReply() functions)
2. Implementation of the sending program (client using the MsgSend() function)
3. Executing programs on the local node (OS in VM)
4. Modification of the program so that:
   - client asks the user to type in a word (word=no space inside) and sends it to the server
   - server prints that word,
     reverses all the characters,
     adds a number in front of reversed word with length of the word,
     replies back to the client with reversed word
   - if the word is ‘exit’, then client and server end their execution (break out of the infinite loop).

Example client session (console):

```
Client application ex21_clientUser17733209673864 is starting...
name_open(ex21_reverser) succeeded, my pid=1404951
Enter a word: abrakadabra
 - calling MsgSend..
 - msgReply.word = 11:arbadakarba, status=11
Enter a word: exit
 - calling MsgSend..
 - msgReply.word = 4:tixe, status=4
Client exit.
```

Example server session (console):

```
Server ex21_serverGHayduk17733209609413 application is starting (name_attach to
<ex21_reverser>)
 - name_attach succeeded, my pid=1404949
MsgReceive is waiting for a message...
 - received abrakadabra, replying with 11:arbadakarba
MsgReceive is waiting for a message...
 - received exit, replying with 4:tixe
Server exit.
```